BRAINSTER

Objects 2

Brainster Web Development Academy

Table of contents

Constructor functions (prototype objects)

- Last time, we learned how we can create objects, assign properties (variables) and methods (functions) to them.
- What if, for example, we wanted to create a list of cat objects and all of them need to share the same code (such as setName or setColor)? It wouldn't be too practical to redefine the same method over and over again.
- One might say we would need a **prototype** for such an occasion...

Prototype objects

- On a theoretical level, JavaScript is a prototype-based language. In simpler terms, that means that every object has a parent object which it inherits methods and properties from.

- That's quite convenient if we'd, for example, like to share the methods mentioned in the previous slide (setName and setColor).

Prototype objects

- JavaScript is a prototype-based language. Practically, that means that every object has a “parent” object which it receives methods and properties from.
- In our example, that’s quite convenient if we like to share the methods mentioned in the previous slide (setName and setColor).

```
function Cat() {
        this.name = ‘’;
        this.setName = function(nameArg) {
                this.name = nameArg;
        }
}
```

```
const cat1 = new Cat();
cat1.setName(‘Thomas’);
console.log(cat1.name); // Thomas
```

```
const cat2 = new Cat();
cat2.setName(‘Paws’);
console.log(cat2.name); // Paws
```


Aside: vocabulary

- In the previous example, the Cat function is called a **constructor function**. That's because it creates a new objects with some given parameters.

- cat1 and cat2 can have multiple names, but the most commonly use one across programming languages is an **instance**.

Exercise 1

1. Define an constructor function called “Cat”.
2. The “Cat” constructor should also contain two properties called “name” and “color”.
3. Create a new cat (make a new instance from the constructor function).
4. Set its name and color as seen on the previous slide. (by sending them when creating the new instance)
5. Log the resulting cat to the console.

Input arguments

- Just as we did with regular functions, we can also set input arguments to constructor functions.

- This is useful if we want to always set a predefined list of input arguments to the instance object - in other words, send name and color as input arguments, instead of calling two separate functions to do it.

```
function Cat(nameArg, colorArg) {
        this.name = nameArg;
        this.color = colorArg;
}

const cat1 = new Cat('Thomas', 'blue');
const cat2 = new Cat('Paws', 'black');

console.log(cat1); // { name: 'Thomas', color: 'blue'}
console.log(cat2); // { name: 'Paws', color: 'black'}
```


Exercise 2

1. Define a method called setName, which receives a string as an input argument and sets that string as the cat's name.
2. Define a method called setColor, which receives a string as an input argument and sets that string as the cat's color
3. Create a new method of the Cat object prototype called "sayNameAndColor" which alerts the name and color of the cat.
4. Create two new cat instances (for example "Thomas" and "blue", "Brenda'' and " black '')
5. Invoke the sayNameAndColor method of the both Cat instances.

Exercise 3

1. Create a constructor function for a Cake object. The function should take in arguments of flavor, price and occasion, and it should save them as properties of the object.
2. Create a method on the Cake object called describe(). When that method is called, it should use console.log to output a string like "The birthday cake has a cherry flavor and costs 30$.“
3. Instantiate a new Cake object and call the describe() method on it.
4. Somehow, it received the wrong occasion value, so you need to change it using dot notation.
5. Call the describe() method again, now the occasion must be changed.

Exercise 4

1. Create the constructor function for a Onlinebook object. The function should take in arguments of title (a string), uploader (a string, the person who uploaded it), and pages (a number of pages), and it should save them as properties of the object.
2. Create a method on the Onlinebook object called read(). When that method is called, it should use console.log to output a string like "You read all 60 pages of Otters Holding Hands which is uploaded by cynthia holmes!“
3. Instantiate two new Onlinebook objects with different constructor arguments and call the read() method on them.
4. Extra: Use an array of data objects and a forEach loop to instantiate 5 Onlinebook objects.

A bit of theory

- In other languages, what we did so far is called “**classes**”.
- Classes represent an abstraction of an object, while the object (instance) is the usable part.
- Translated to our previous example, “Cat” would be the class - by itself it does nothing. “cat1” and “cat2” would be objects (instances) of the “Cat” class.
- We can change the properties and methods of “cat1” and “cat2” just as with any other object.